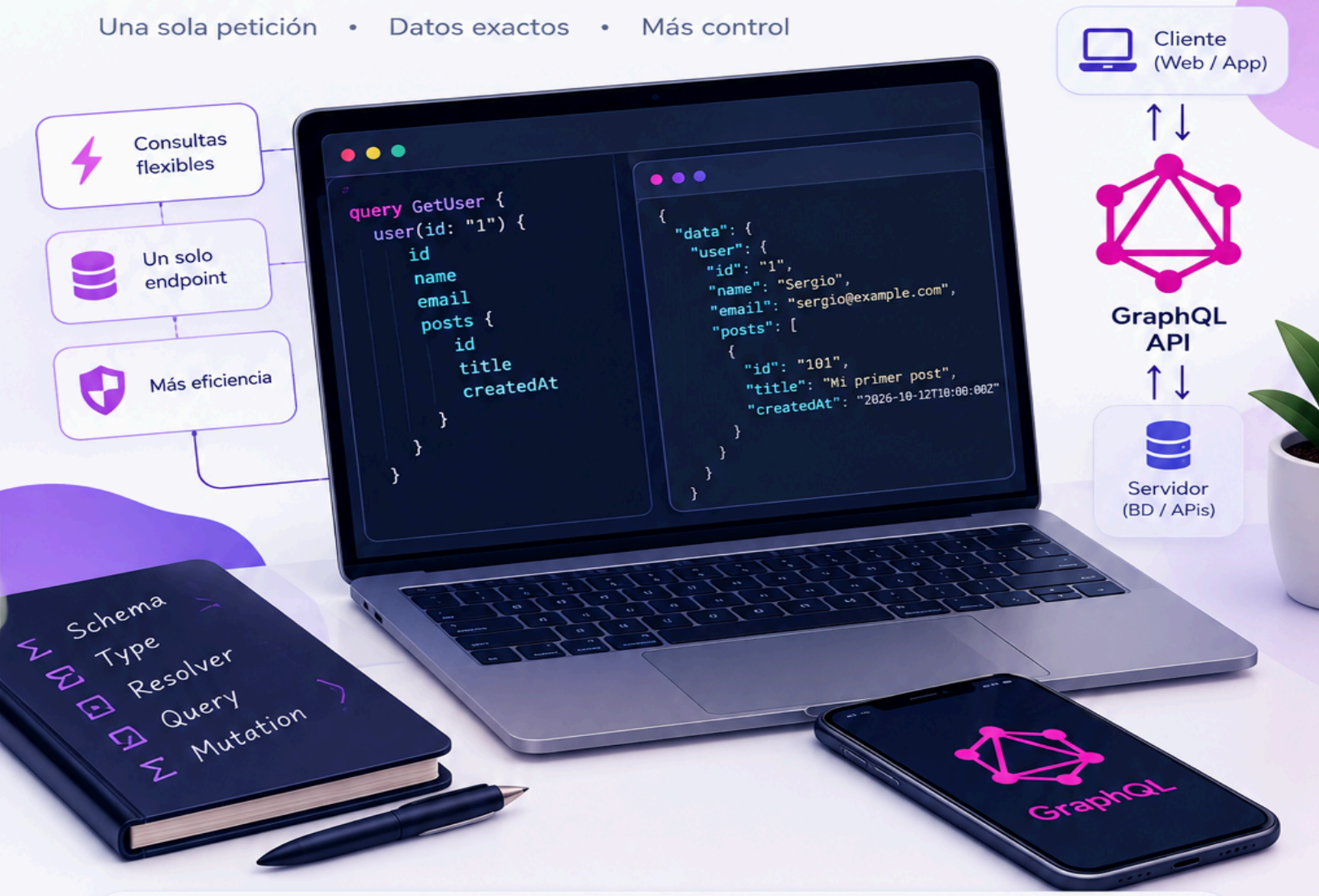
MANUAL

GraphQL

Consulta, manipula y optimiza tus datos con GraphQL

Una sola petición • Datos exactos • Más control



Alumno: Sergio Vilchez Camuñas

Curso: 2º DAW

Asignatura: Optativa-Desarrollo Frontend

Fecha: 12 de octubre de 2026

ÍNDICE

1. ¿Qué es y para qué sirve?.....3
a. Definición con tus propias palabras.....3
b. Qué problema resuelve y por qué surgió.....3
c. Quién la crea o mantiene y versión actual.....3
2. Conceptos clave.....4
3. Requisitos e instalación.....6
a. Requisitos previos.....6
b. Comandos de instalación.....7
c. Estructura de carpetas generada.....12
d. Demostración de funcionamiento.....12
4. Ejemplo básico de uso.....14
5. Ventajas, inconvenientes y conclusión.....19
6. Recursos y bibliografía.....20

1. ¿Qué es y para qué sirve?

a. Definición con tus propias palabras.

GraphQL es un lenguaje de consultas para APIs que permite pedir los datos que se necesiten para una aplicación en una consulta y el servidor los devuelve normalmente en un JSON con la misma estructura en un único “endpoint” (La URL de cada recurso en una API donde un cliente se conecta al servidor para enviar o recibir datos).

b. Qué problema resuelve y por qué surgió.

En una API REST el servidor devuelve los datos de una forma fija en cada respuesta, lo que provoca:

- ***Over-Fetching:*** El endpoint devuelve más campos de los que se piden, por ejemplo pides el nombre de un usuario pero te devuelve el objeto usuario completo, consumiendo más recursos.
- ***Under-Fetching:*** Es un posible problema cuando en un único endpoint no tienes todos los datos que necesitas así que tienes que hacer varias peticiones, por ejemplo para obtener un usuario, los post de un usuario, etc.

En GraphQL el cliente describe en una consulta los campos que necesita incluidos los relacionados.

No sustituye siempre a las peticiones REST porque añade complejidad en el servidor y no es siempre la mejor opción pero ayuda a resolver problemas concretos.

c. Quién la crea o mantiene y versión actual.

GraphQL se originó en Facebook en 2012 por la necesidad que tienen con sus aplicaciones móviles, donde optimizar las peticiones importa mucho. En 2015 se publicó como código abierto y actualmente lo mantiene la GraphQL Foundation. La versión actual de la especificación es la de septiembre de 2025 y la de la implementación (como la librería GraphQL.js) es la versión 17.0.2 de julio de 2026.

2. Conceptos clave

- ***Schema y types:*** Describe los datos que existen y de qué tipo es cada uno. Se escriben en "Schema Definition Language" (SDL).
  Los tipos escalares básicos que tiene son: String, Int, Float, Boolean y ID.
  Si le añades "!" indicas que el campo no puede ser nulo.

```
type Videojuego {
  id: ID!
  titulo: String!
  plataforma: String!
  anio: Int
  completado: Boolean!
}
```

- ***Query:*** Es la operación para pedir los datos. El cliente escribe los datos que quiere y el servidor lo devuelve en JSON. Para pedir un dato concreto se añaden argumentos.

```
query {
  videojuego(id: "1") {
    titulo
    anio
  }
}
```

- ***Mutation:*** Es la operación para modificar datos. Tiene la misma sintaxis que query y devuelve el dato modificado directamente para no tener que volver a hacer una consulta.

```
mutation {
  anadirVideojuego(titulo: "Hollow Knight", plataforma: "PC") {
    id
    titulo
  }
}
```

- ***Resolver:*** Es la función del servidor que dice como se obtiene cada dato, hay una por cada campo de query y mutation. Reciben cuatro parámetros en orden: “parent”, “args”, “contextValue" e “info”.

```
videojuego: (parent, args) => videojuegos.find((v) => v.id === args.id)
```

- Variables: Se usan para pasar valores a una consulta sin escribirlos dentro del texto. Se declaran con “$” y se envían en un JSON aparte.

```
query ObtenerJuego($id: ID!) {
  videojuego(id: $id) {
    titulo
  }
}
```

3. Requisitos e instalación

a. Requisitos previos

Necesitaremos lo siguiente:

- ***Node.js:*** Permite ejecutar JavaScript fuera del navegador y poner en funcionamiento el servidor.
- ***NPM:*** Permite instalar las bibliotecas necesarias.
- ***Visual Studio Code:*** Entorno de desarrollo para hacer el código.
- ***Git:*** Lo necesitamos para poder subir la práctica a GitHub.
- ***Cuenta GitHub:*** Para poder subir el proyecto y tenerlo en nuestro repositorio.
- ***Navegador:*** El que queramos para las pruebas.

```
sergi@PC-Sergio UCRT64 /c/Sergio/2DAW/Optativa/Manual-GraphQL-Vilchez (main)
$ node -v
v24.21.0

sergi@PC-Sergio UCRT64 /c/Sergio/2DAW/Optativa/Manual-GraphQL-Vilchez (main)
$ npm -v
11.19.0

sergi@PC-Sergio UCRT64 /c/Sergio/2DAW/Optativa/Manual-GraphQL-Vilchez (main)
$ git --version
git version 2.56.0.windows.1
```



b. Comandos de instalación

Creamos la carpeta de ejemplo con el comando “mkdir” y accedemos a ella. Luego inicializamos el proyecto con “npm init -y” que nos creará el archivo “package.json” que describe el proyecto y las dependencias.

```
sergi@PC-Sergio UCRT64 /c/Sergio/2DAW/Optativa/Manual-GraphQL-Vilchez (main)
$ mkdir ejemplo

sergi@PC-Sergio UCRT64 /c/Sergio/2DAW/Optativa/Manual-GraphQL-Vilchez (main)
$ cd ejemplo/

sergi@PC-Sergio UCRT64 /c/Sergio/2DAW/Optativa/Manual-GraphQL-Vilchez/ejemplo
(main)
$ npm init -y
Wrote to C:\Sergio\2DAW\Optativa\Manual-GraphQL-Vilchez\ejemplo\package.json:

{
  "name": "ejemplo",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

Ahora ajustamos package.json para que podamos usar import, lo que nos exige cambiar el tipo a “module”. Y con el segundo comando añadimos un script para poder iniciar la aplicación simplemente con el comando “npm start”.

```
sergi@PC-Sergio UCRT64 /c/Sergio/2DAW/Optativa/Manual-GraphQL-Vilchez/ejemplo (main)
$ npm pkg set type="module"

sergi@PC-Sergio UCRT64 /c/Sergio/2DAW/Optativa/Manual-GraphQL-Vilchez/ejemplo (main)
$ npm pkg set scripts.start="node index.js"

sergi@PC-Sergio UCRT64 /c/Sergio/2DAW/Optativa/Manual-GraphQL-Vilchez/ejemplo (main)
$ cat package.json
{
  "name": "ejemplo",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1",
    "start": "node index.js"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "module"
}
```

Comprobamos la compatibilidad antes de instalar. Como podemos ver la última versión de Apollo Server es la 5.5.1 y con el segundo comando nos indica que es compatible con GraphQL con la versión 16.11 o superior, pero menor que 17.

```
sergi@PC-Sergio UCRT64 /c/Sergio/2DAW/Optativa/Manual-GraphQL-Vilchez/ejemplo (main)
$ npm view @apollo/server version
5.5.1

sergi@PC-Sergio UCRT64 /c/Sergio/2DAW/Optativa/Manual-GraphQL-Vilchez/ejemplo (main)
$ npm view @apollo/server peerDependencies
{ graphql: '^16.11.0' }
```



A continuación vamos a instalar Apollo Server y GraphQL, para ello usamos el siguiente comando indicando la versión para que sean compatibles y no de error.

```
sergi@PC-Sergio UCRT64 /c/Sergio/2DAW/Optativa/Manual-GraphQL-Vilchez/ejemplo (main)
$ npm install @apollo/server graphql@16
```

Y comprobamos que se han instalado de forma correcta. Usamos el siguiente comando y observamos las versiones. También comprobamos que se ha actualizado “package.json” y que ahora tiene las dos librerías.

```
sergi@PC-Sergio UCRT64 /c/Sergio/2DAW/Optativa/Manual-GraphQL-Vilchez/ejemplo (main)
$ npm ls
ejemplo@1.0.0 C:\Sergio\2DAW\Optativa\Manual-GraphQL-Vilchez\ejemplo
├── @apollo/server@5.5.1
└── graphql@16.14.2

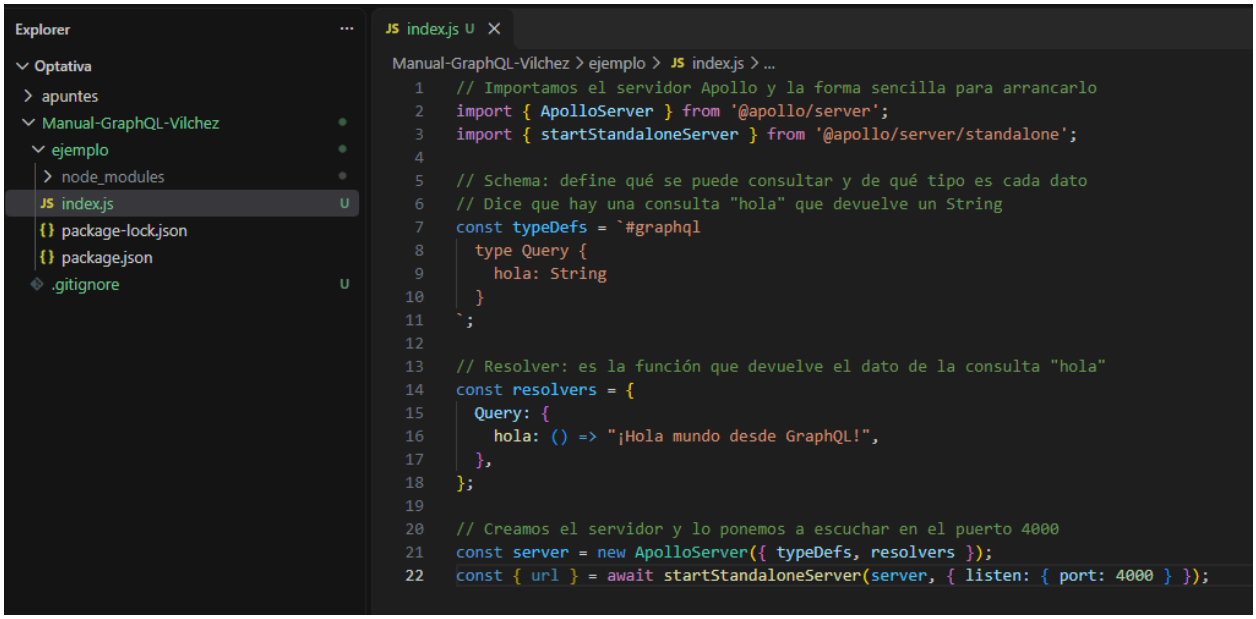

sergi@PC-Sergio UCRT64 /c/Sergio/2DAW/Optativa/Manual-GraphQL-Vilchez/ejemplo (main)
$ cat package.json
{
  "name": "ejemplo",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1",
    "start": "node index.js"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "module",
  "dependencies": {
    "@apollo/server": "^5.5.1",
    "graphql": "^16.14.2"
  }
}
```

Como paso opcional recomendado vamos a hacer que “node_modules” no se suba a GitHub puesto que pesa mucho y se regenera cuando hacemos el comando npm install. Para ello volvemos a la carpeta raíz del repositorio y creamos un archivo “.gitignore” con la línea “node_modules”.

```
sergi@PC-Sergio UCRT64
/c/Sergio/2DAW/Optativa/Manual-GraphQL-Vilchez/ejemplo (main)
$ cd ..

sergi@PC-Sergio UCRT64 /c/Sergio/2DAW/Optativa/Manual-GraphQL-Vilchez (main)
$ echo node_modules>.gitignore
```

Para terminar con la instalación vamos a crear el archivo “index.js” en la carpeta ejemplo que creamos previamente.



```
// Importamos el servidor Apollo y la forma sencilla para arrancarlo
import { ApolloServer } from '@apollo/server';
import { startStandaloneServer } from '@apollo/server/standalone';

// Schema: define qué se puede consultar y de qué tipo es cada dato
// Dice que hay una consulta "hola" que devuelve un String
const typeDefs = `#graphql
  type Query {
    hola: String
  }
`;

// Resolver: es la función que devuelve el dato de la consulta "hola"
const resolvers = {
  Query: {
    hola: () => "¡Hola mundo desde GraphQL!",
  },
};

// Creamos el servidor y lo ponemos a escuchar en el puerto 4000
const server = new ApolloServer({ typeDefs, resolvers });
const { url } = await startStandaloneServer(server, { listen: { port: 4000
} });
```

Este servidor es uno básico simplemente para poder comprobar que la instalación funciona.

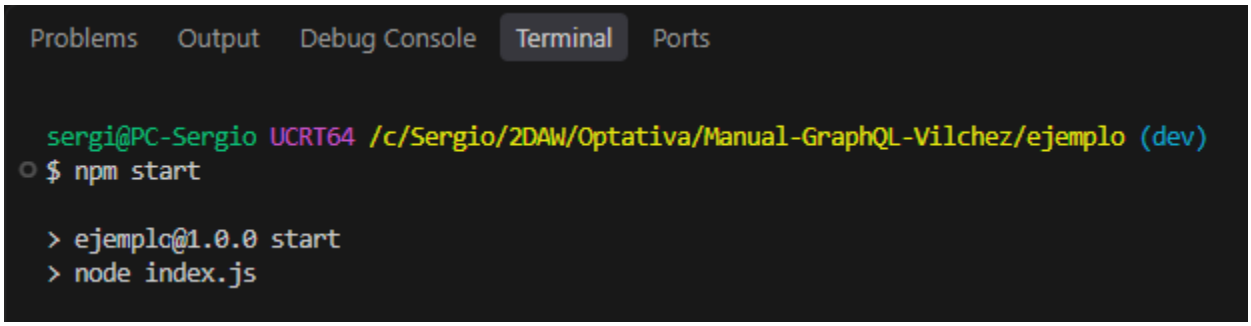
C. Estructura de carpetas generada

```
sergi@PC-Sergio UCRT64 /c/Sergio/2DAW/Optativa/Manual-GraphQL-Vilchez (dev)
$ npx tree-node-cli -a -I ".git|node_modules/"
Manual-GraphQL-Vilchez
└── ejemplo
    ├── index.js
    ├── package-lock.json
    └── package.json
```

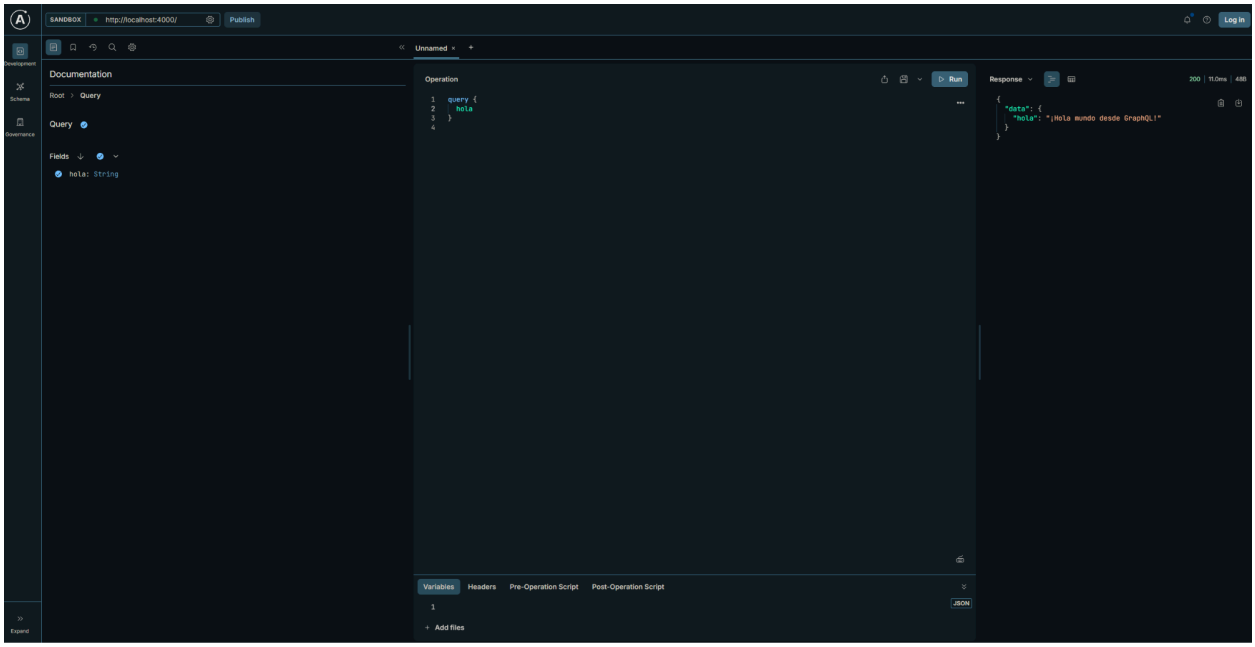
- **Index.js ->** Nuestro código para el servidor de prueba.
- **package-lock.json ->** Guarda las versiones instaladas para que se pueda reproducir en otro equipo.
- **package.json ->** Describe el proyecto con sus dependencias y scripts.

d. Demostración de funcionamiento

Dentro de ejemplo hacemos un “npm start”. Como habíamos configurado en package.json que arrancase index.js el script se ejecutará y arrancará el servidor. Si windows pide permisos le daremos que si.



Accedemos a http://localhost:4000/ y nos aparecerá el sandbox de Apollo. Borramos el ejemplo que nos da y hacemos la consulta para “hola” que hemos creado nosotros. Le daremos a “run” y veremos que nos devuelve el String que había en hola.




```
query {
  hola
}
```

4. Ejemplo básico de uso

Vamos a hacer una simulación de un pequeño catálogo de videojuegos. Para ello modificamos nuestro “Index.js”.

```
// Importamos el servidor Apollo y la forma sencilla para arrancarlo
import { ApolloServer } from '@apollo/server';
import { startStandaloneServer } from '@apollo/server/standalone';

// type Videojuego -> definimos como van a ser los objetos videojuego dando a
cada campo el nombre y el tipo de este. Si tenemos ! indica que el campo no puede
ser nulo.
// type Query -> definimos lo que se puede leer. Videojuegos devuelve una lista
con los juegos y videojuego devuelve únicamente el juego que pidamos
// type Mutation -> definimos lo que se puede modificar. Tenemos una operación
que recibe como parámetros título y plataforma y devuelve el juego creado.
const typeDefs = `#graphql
    type Videojuego {
        id: ID!
        titulo: String!
        plataforma: String!
        anio: Int
        completado: Boolean!
    }

    type Query {
        videojuegos: [Videojuego!]!
        videojuego(id: ID!): Videojuego
    }

    type Mutation {
        anadirVideojuego(titulo: String!, plataforma: String!): Videojuego!
    }
`;

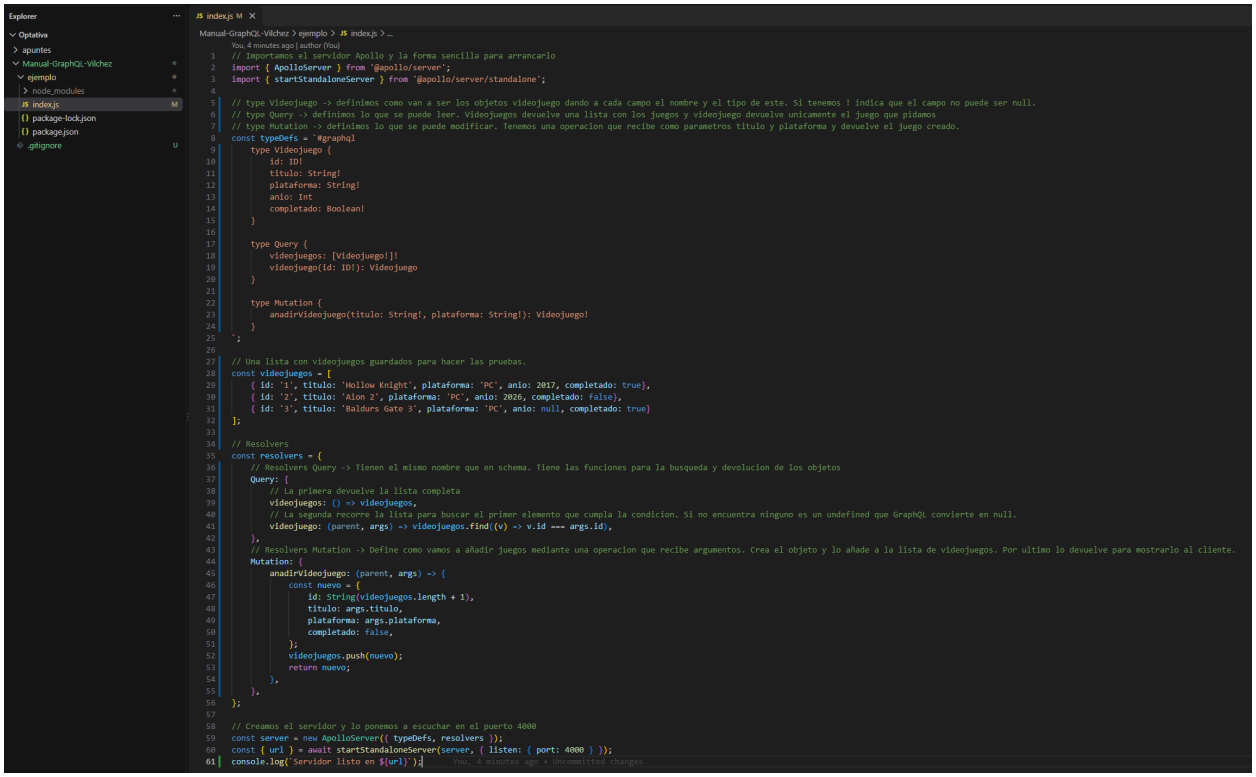
// Una lista con videojuegos guardados para hacer las pruebas.
const videojuegos = [
```



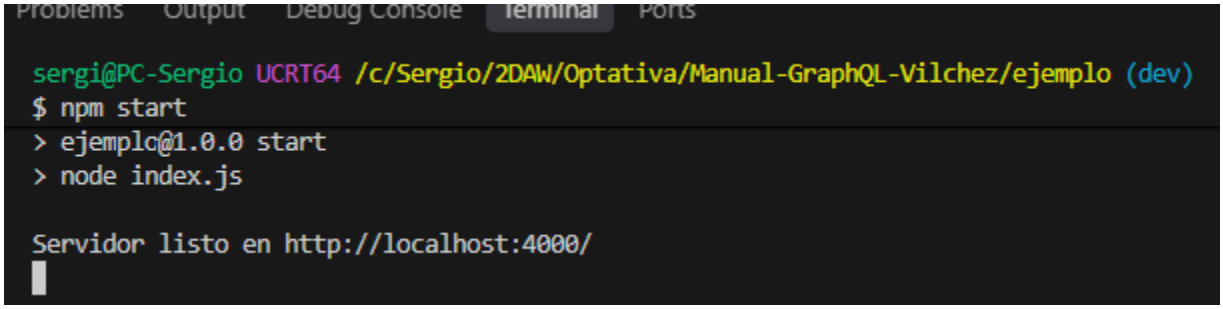

```
    { id: '1', titulo: 'Hollow Knight', plataforma: 'PC', anio: 2017, completado:
true},
    { id: '2', titulo: 'Aion 2', plataforma: 'PC', anio: 2026, completado:
false},
    { id: '3', titulo: 'Baldurs Gate 3', plataforma: 'PC', anio: null,
completado: true}
];
// Resolvers
const resolvers = {
    // Resolvers Query -> Tienen el mismo nombre que en schema. Tiene las
funciones para la búsqueda y devolución de los objetos
    Query: {
        // La primera devuelve la lista completa
        videojuegos: () => videojuegos,
        // La segunda recorre la lista para buscar el primer elemento que cumpla
la condición. Si no encuentra ninguno es un undefined que GraphQL convierte en
null.
        videojuego: (parent, args) => videojuegos.find((v) => v.id === args.id),
    },
    // Resolvers Mutation -> Define cómo vamos a añadir juegos mediante una
operación que recibe argumentos. Crea el objeto y lo añade a la lista de
videojuegos. Por último lo devuelve para mostrarlo al cliente.
    Mutation: {
        anadirVideojuego: (parent, args) => {
            const nuevo = {
                id: String(videojuegos.length + 1),
                titulo: args.titulo,
                plataforma: args.plataforma,
                completado: false,
            };
            videojuegos.push(nuevo);
            return nuevo;
        },
    },
};
```

```
// Creamos el servidor y lo ponemos a escuchar en el puerto 4000
const server = new ApolloServer({ typeDefs, resolvers });
const { url } = await startStandaloneServer(server, { listen: { port: 4000 } });
console.log(`Servidor listo en ${url}`);
```

Captura del código en Visual Studio Code.

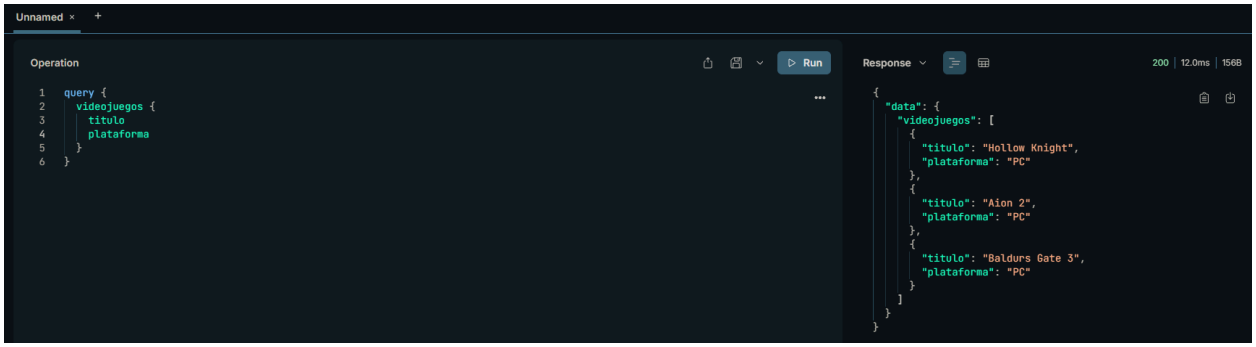


Ahora arrancamos el servidor como antes.

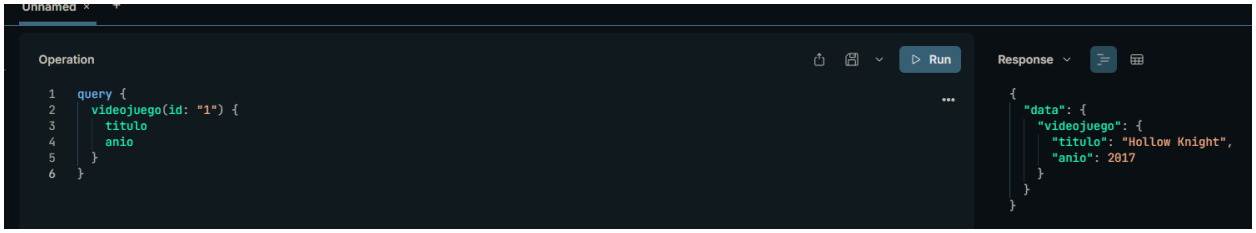


Y podemos hacer pruebas como las siguientes:

Consultar el título y la plataforma de los juegos que tenemos en la lista.

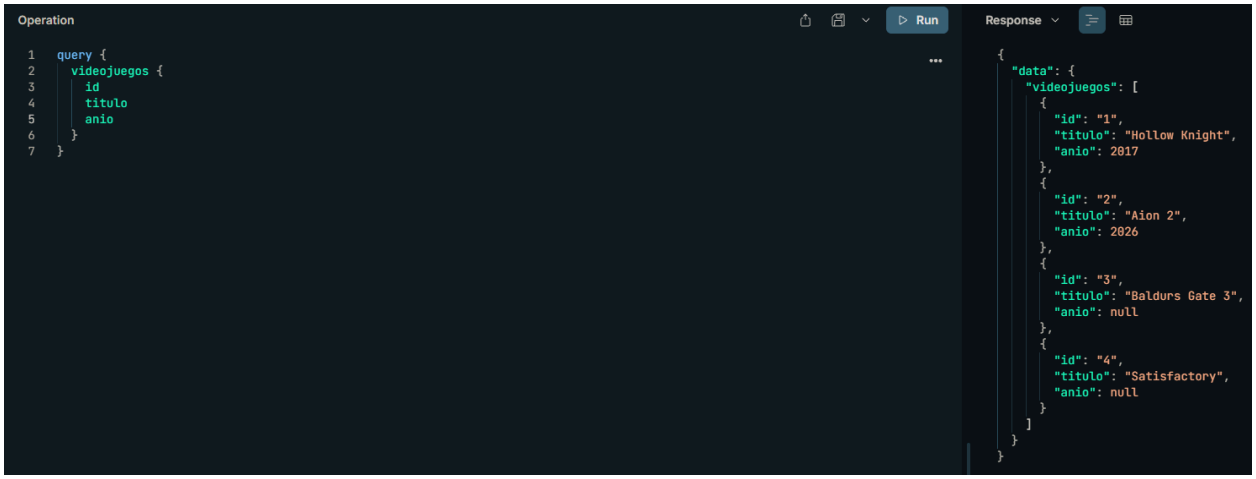


Consultar el título y año de un videojuego en concreto.



Añadimos un nuevo videojuego.

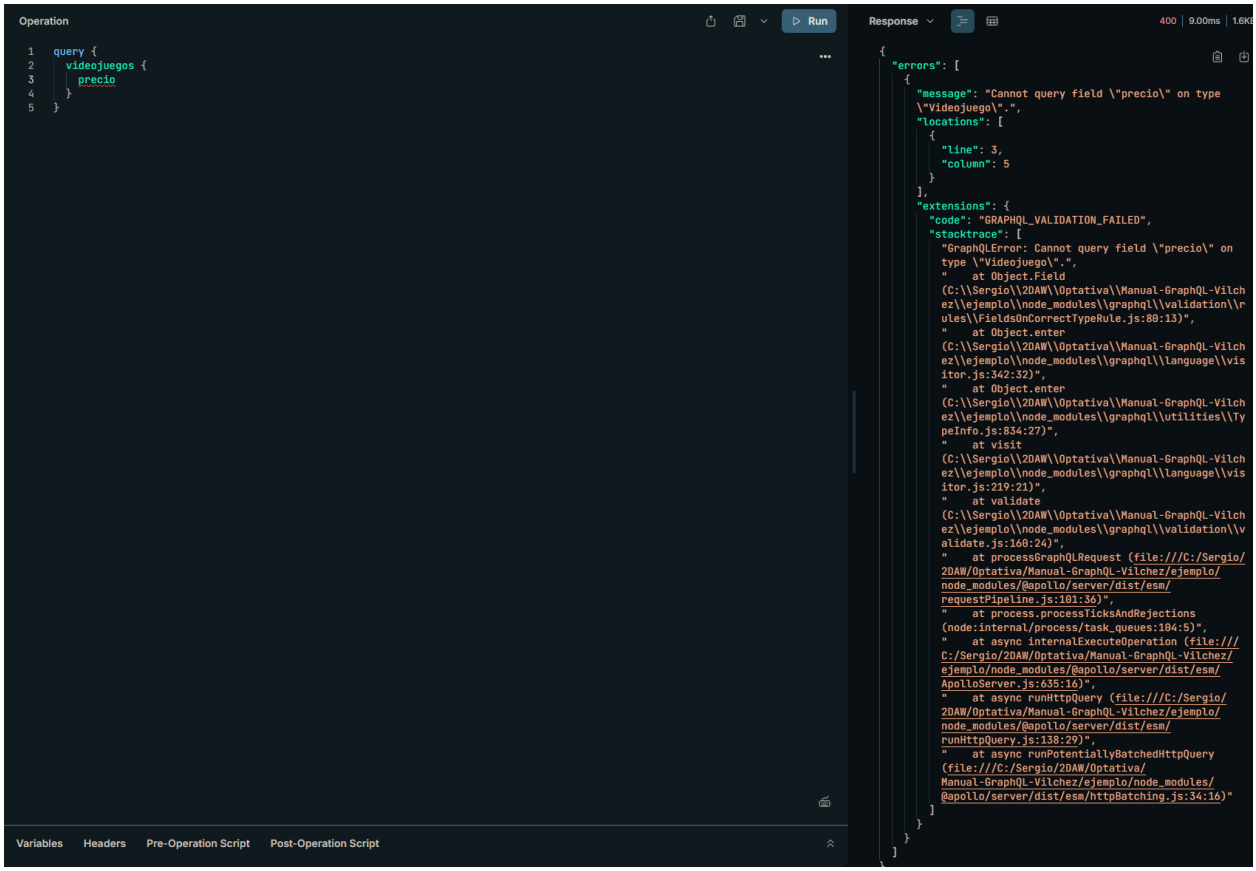




Ahora con variables que podemos definir abajo del sandbox de Apollo.



Por último una prueba pidiendo un campo que no existe. Dará error al no existir en el schema y intentar hacer la validación.



5. Ventajas, inconvenientes y conclusión

Ventajas:

- Se piden los datos que se necesitan solamente, evitando el Over-Fetching.
- Solo se necesita un endpoint para todas las operaciones a diferencia de las API REST.
- Se válida de forma automática.
- Tiene un tipado fuerte lo que reduce errores.

Inconvenientes:

- Hay que aprender el lenguaje SDL y cómo funcionan los resolvers, queries y mutations.
- Hay más complejidad en el servidor al estar todo junto en un único endpoint.
- Hay peligro de que se hagan peticiones anidadas que sean muy pesadas y carguen mucho al servidor.
- Para poder utilizarlo de forma real para hacer consultas requiere que el servidor sea compatible con GraphQL.

Conclusión:

Es una herramienta muy útil. En un principio pensaba que me permitiría hacer peticiones a APIs de forma más “simple”, sin tener que filtrar los datos que quisiera de todo lo que devuelve una API REST pero cuando he investigado me he dado cuenta que si bien tiene sus ventajas, sobre todo para el ámbito de aplicaciones móviles que al final fue su punto de origen y el porque lo desarrollaron, también tiene sus desventajas y requiere que la API sea compatible. Es verdad que hay que aprender bastantes conceptos que yo he tocado y mirado de forma simple para la práctica y que si quisiera usarla mejor tendría que profundizar en ellos pero una vez entiendes como funciona parece bastante cómoda de usar, al menos para realizar las peticiones, la configuración del lado del servidor es más compleja.. Además puedes aprender bastante en la página web oficial.
¿Lo usaría en un proyecto real? pues quizá para una aplicación móvil pero después de lo que he investigado creo que para una aplicación web optaria por REST.

6. Recursos y bibliografía

- GraphQL.org -> Página oficial de GraphQL que contiene un montón de recursos y tutoriales que explican cómo funciona el lenguaje y en el que incluso se pueden hacer pruebas simples.

- ApolloGraphQL.com -> Página oficial de Apollo Server con explicaciones para instalarlo, definir los schema, resolvers y listas para hacer pruebas. Además de como iniciar el server y hacer las querys en el sandbox.

- Youtube -> Videos explicando GraphQL para principiantes, como el que enlazo. Al principio para entender cosas de forma simple que no entendía ni leyendo la página oficial me han resultado muy útiles.

- Claude -> IA para revisar el código y que me ayudara a entender conceptos y solucionar problemas, lo añado porque lo he usado para lo que se permite en la práctica y porque lo he tenido que utilizar.

REPOSITORIO DE GITHUB